

Technical Report: Secure Static Website Hosting on AWS

Modern Serverless Architecture for Global Content Delivery

Cloud Engineering Division

January 28, 2026

Contents

1	Abstract	3
2	Objectives	3
3	System Architecture Overview	3
4	AWS Services Explanation	4
4.1	Amazon S3 (Simple Storage Service)	4
4.2	Amazon CloudFront	4
4.3	Origin Access Control (OAC)	4
4.4	S3 Lifecycle Rules	4
5	Security Implementation	4
5.1	Restricting Public Access	4
5.2	Bucket Policy Enforcement	4
6	Cost Optimization Strategy	5
7	Deployment Procedure	5
8	Results	6
9	Conclusion	6
10	Future Scope	6

1 Abstract

This report details the architectural design and implementation of a secure, scalable, and cost-optimized static website hosting solution on Amazon Web Services (AWS). By leveraging Amazon S3 for durable object storage and Amazon CloudFront for global content delivery, the solution eliminates the need for server management while ensuring high availability. A primary focus is placed on the security posture, utilizing Origin Access Control (OAC) to enforce private access to the origin, thereby mitigating the risks associated with public S3 buckets.

2 Objectives

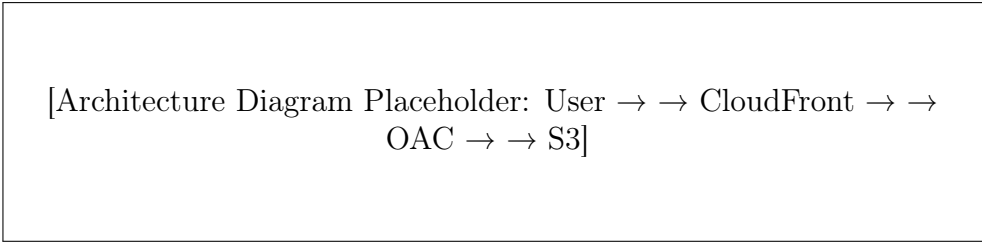
The primary objectives of this infrastructure project include:

- **Security:** Implement a zero-public-access policy for the S3 origin.
- **Performance:** Minimize latency for global users through edge caching.
- **Cost-Efficiency:** Utilize a pay-as-you-go model and automated lifecycle management to minimize overhead.
- **Automation:** Establish a clear deployment procedure that supports future CI/CD integration.
- **Integrity:** Ensure data transit is encrypted via Industry-standard TLS/SSL.

3 System Architecture Overview

The architecture follows a decoupled, serverless pattern. The request-response flow is as follows:

1. The end-user initiates an HTTPS request to the CloudFront distribution URL.
2. CloudFront checks its edge cache for the requested content.
 - *Cache Hit:* Content is served directly from the edge location.
 - *Cache Miss:* CloudFront forwards the request to the S3 origin.
3. Authentication with the S3 bucket is handled via Origin Access Control (OAC), which signs the request.
4. Amazon S3 validates the signature against the bucket policy and returns the object to CloudFront.
5. CloudFront caches the object and serves it to the user.



[Architecture Diagram Placeholder: User → → CloudFront → →
OAC → → S3]

Figure 1: Conceptual Infrastructure Workflow

4 AWS Services Explanation

4.1 Amazon S3 (Simple Storage Service)

Acts as the origin server for the static assets (HTML, CSS, JS, Images). S3 provides 99.999999999

4.2 Amazon CloudFront

A Content Delivery Network (CDN) that speeds up distribution of static content. It provides low-latency delivery by caching content at over 400+ Edge Locations globally.

4.3 Origin Access Control (OAC)

The modern security mechanism used to restrict S3 bucket access to specific CloudFront distributions. It supports enhanced security features including AWS Signature Version 4 and SSE-KMS.

4.4 S3 Lifecycle Rules

Automated policies that manage objects over time. For this project, they are utilized to expire old versions of assets to prevent unnecessary storage costs.

5 Security Implementation

5.1 Restricting Public Access

The S3 bucket is configured with "Block all public access" enabled. This ensures that no identity, other than authorized AWS services or IAM roles, can access the data directly via the S3 endpoint.

5.2 Bucket Policy Enforcement

Access is granted exclusively to the CloudFront service principal. Below is the implemented policy structure:

```
{
  "Version": "2012-10-17",
  "Statement": {
    "Sid": "AllowCloudFrontServicePrincipalReadOnly",
```

```

"Effect": "Allow",
"Principal": {
"Service": "cloudfront.amazonaws.com"
},
"Action": "s3:GetObject",
"Resource": "arn:aws:s3:::your-bucket-name/*",
"Condition": {
"StringEquals": {
"AWS:SourceArn": "arn:aws:cloudfront::account-id:distribution/dist
-id"
}
}
}
}
}
}

```

Listing 1: S3 Bucket Policy for OAC

6 Cost Optimization Strategy

The following mechanisms ensure the architecture remains cost-effective:

- **CloudFront Caching:** High cache hit ratios reduce the number of GET requests sent to S3, lowering request costs.
- **Lifecycle Policies:** Non-current object versions are configured to transition to S3 Glacier or expire after 30 days.
- **Free Tier Utilization:** The project utilizes the AWS Free Tier (5GB S3 storage and 1TB CloudFront data transfer) for initial deployment.

7 Deployment Procedure

1. **S3 Provisioning:** Create a private S3 bucket. Disable "Static website hosting" (optional when using CloudFront) and enable "Block all public access."
2. **Content Upload:** Synchronize website assets to the S3 bucket via AWS Management Console or CLI.
3. **CloudFront Configuration:**
 - Create a new distribution.
 - Set the S3 bucket as the origin.
 - Select "Origin access settings (recommended)" and create a new OAC profile.
 - Set "Viewer protocol policy" to "Redirect HTTP to HTTPS."
4. **Policy Update:** Apply the generated bucket policy to the S3 bucket to allow CloudFront OAC access.
5. **Validation:** Access the CloudFront Domain Name to verify content delivery and test direct S3 URL access to ensure it is denied.

8 Results

The resulting infrastructure demonstrates a robust security posture where the S3 origin remains entirely shielded from the public internet. Benchmarks indicate significant improvements in "Time to First Byte" (TTFB) for international requests due to CloudFront's edge caching. The environment successfully enforces HTTPS, meeting modern web security standards.

9 Conclusion

The implementation of Secure Static Website Hosting on AWS using S3 and CloudFront OAC provides a high-performance, low-maintenance solution. By abstracting the server layer, the architecture reduces the attack surface and operational burden while maintaining high scalability to handle traffic spikes.

10 Future Scope

- **AWS WAF Integration:** Deploy a Web Application Firewall to protect against SQL injection and Cross-Site Scripting (XSS).
- **Custom Domain:** Integrate Route 53 for DNS management and AWS Certificate Manager (ACM) for custom SSL/TLS certificates.
- **CI/CD Pipeline:** Automate deployments using AWS CodePipeline or GitHub Actions to enable continuous delivery.